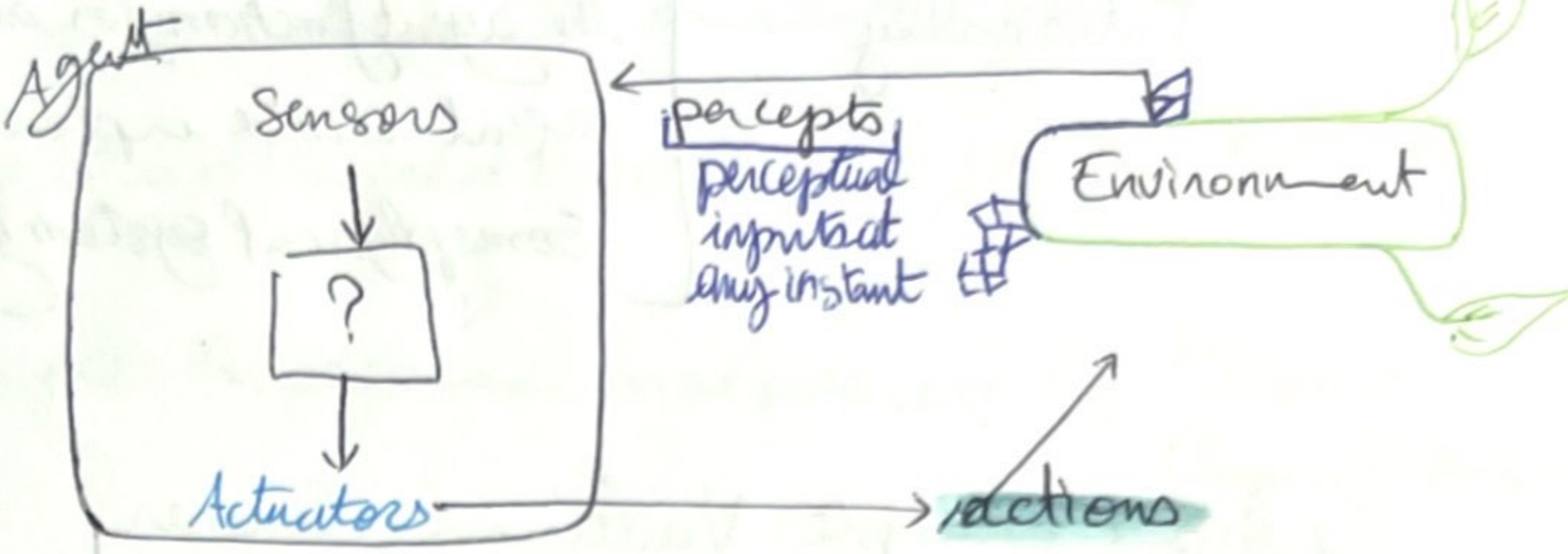
Intelligent Agents:

Rational Agents central to our approach to artificial intelligence

* **Agent**: anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators



Examples

① Human agent

Sensors: eyes, ears, hands...

Actuators: hands, legs, vocal tract, ...

② Robotic agent

Cameras, infrared and sonar range finders

various motors, robot arm...

③ Software agent

Keystrokes, file contents, network packets

displaying on the screen, writing on files, sending network packets.

* **Agent percept sequence**: complete history of everything the agent has ever perceived.

• Choice of Action: at any given instant, can depend on the entire sequence observed to date, but not on anything it HAS NOT perceived.

Mathematically speaking: agent's behaviour is described by the agent function that maps any given percept sequence to an action

AGent Function: Percept sequence $\longrightarrow$ Action.

The agent function:

Externally: pictured as a table of mappings between percept sequences and actions.
Such table is an external (abstract mathematical) characterization of the agent function.

Internally: the agent function for an artificial agent will be implemented within some physical system by an agent program.

- Agent example: Vaccum-cleaner.
  1. percieves the square it is in / dirty place
  2. Actions: move left / right / suck up dirt / nothing.
  3. Partial tabulation of its agent function. page 9

WE NEED to: • define the Actions corresponding to the percepts

•

Unlike all areas of engineering: AI operates where artifacts have significant computational resources and the task environment requires nontrivial decision making

Good Behaviour: Concept of Rationality:

Rational Agent is the one that does the right thing
(every action in the table is correct)

Obvious approach: consider the consequences of behaviours

percepts ⟶ sequences of actions ⟶ environment goes through a sequence of states.

The performance measure evaluates any given sequence of environment states.

- it is designed specifically for the task at stake

→ In our example: the performance measure can be Reward/Penality (for noise)

General Rule: design performance measures according to expectations in the environment.

Rationality: Depends on 4 factors:

- Performance measure that defines the Criterion of success
- The agent's prior knowledge of the environment
- The actions that the agent can perform.
- The agent's percept sequence to date.

DEFINITION of Rational Agent: For each percept sequence the Rational Agent should select an action that is expected to Maximize its Performance Measure (given the evidence provided by the percept sequence and whatever built-in knowledge the agent has)

Remark: the same agent could be irrational under different circumstances.
(for example if the clean place stays clean)

Omniscience, learning and autonomy:

- Omniscient agent: who knows the actual outcome of its actions and can act accordingly
  → Impossible in reality because we cannot anticipate everything.

- Difference between Rationality and Perfection.

Rationality	Perfection:
maximizes expected performance	maximizes ACTUAL performance. (impossible).

- Doing actions in order to modify future percepts (sometimes called information gathering) is an important part of rationality.

Example it is not rational to cross the road with uninformatitive percept sequence (like looking right only)
→ It must learn as much as possible from what it perceives more than gathering info.

☆ The initial configuration of the agent can Reflect some prior knowledge of the environment

↳ but with its experience it can be modified and ↑↑ augmented / if it DOES NOT (and it only relies on its prior knowledge then we say that: the Agent lacks autonomy)

the agent should be!

★ Autonomous Agent: learn what it can to compensate for partial or incorrect prior knowledge.

seldom ≠ frequently often

• Complete autonomy from the start is seldom rare

with little or no experience the agent would have to act randomly unless the designer gave some assistant

It is Reasonable to provide an artificial intelligent agent with some initial knowledge, as well as an ability to learn.

After sufficient experience of its environment

Its Behaviours can become effectively independent of its prior knowledge.

Environments:

- The first step is to specify the task environment as fully as possible

Identify the PEAS:

- Performance Measure: Safe, fast, legal, comfortable trip, maximize profits, ...
- Environment: Roads, other traffic, pedestrians, customers ...
- Actuators: Steering, accelerator, brake, horn, display ...
- Sensors: Cameras, Sonars, speedometer, GPS, odometer, accelerometer, engine sensors, (Some examples in slide 22)

are all the values, variables, known to the agent?

- access to the complete state of the environment

observability

- **Fully observable**
  - Sensors detect all aspects that are relevant to the action. (depends on the performance measure).
- **Partially observable**
  - noisy and inaccurate sensors, parts of the state are missing from the sensor data.

No Sensors $\Rightarrow$ $\boxed{\text{UNOBSERVABLE environment}}$

Single agent Vs MultiAgent

→ operate by itself in the environment

→ It needs more than one agent in the environment.

Must the agent treat others as objects/agents?

Key distinction: Is B's behaviour best described as maximizing a performance measure [for (chess rules) example it minimizes A's performance measure] Whose value depends on agent A's behaviour?

slide 27

Predictibility

Deterministic Vs Stochastic

transition model is deterministic (unique successor state given current state and action).

↳ distribution over successor states given current state and action. (probabilities)

Or Strategic: the environment is deterministic except for the actions of other agents.

- in MOST real situations (they are complex) that is impossible to keep track of all the unobsorved aspects → for practical purposes they must be treated as stochastic.

→ Not deteministic → uncertain or not fully observable.

→ Nondeterministic → actions are characterized by their possible outcomes NOT their probabilities and they are usually associated with performance measures that require the agent to succeed for all possible outcomes of its actions.

program.

Episodic VS sequential:

Simpler → the agent doesnot need to think ahead.

the agent's experience is divided into atomic episodes in each of them the agent receives a percept and then performs a single action.

→ Current decision could affect all future decisions.

Short-term actions → long term consequences

* Episodic

Task structure

* Next episode: doesnot depend on the actions taken in previous episodes. eg: Classification tasks.

stability

Static VS Dynamic

→ World changing while the agent is thinking.

SEMI-dynamic: environment doesnot change but the agent's performance score DOES!

Granularity

Discrete VS Continious

applies to the state of the environment to the way time is handled. and to the percepts and actions of the agent.

Chess has discrete number of percepts and actions.

- actions are continious
- range of continious values over time.

infinite state space.

are the rules of the environment known to the agent?

Known VS Unknown:

(outcome probabilities if it is stochastic.) outcomes for all actions are given

- it can be partially observable.

eg: (Solitaire)

Not the same as fully/partially observable.

- agent will have to learn how it works in order to make good decisions.

Can be fully observable (videogames).

The structure of Agents:

Agent = architecture + program.

AI's aim: design an agent program that maps percept sequences into actions

→ assumed to run on some computing device with physical sensors and actuators. called the architecture.

→ it takes into account the architecture (PC, robot, ..) in the commands it issues... eg: walk, move, ...

- Lookup of a percept sequence into the table of actions: and we need to select the appropriate action for every possible percept sequence.

→ function TABLE_DRIVEN_AGENT(percept) returns Action ←

(but this table-driven approach to agent design is doomed to failure!)

slide 43

↳ huge space complexity! $\sum_{t=1}^{T} |P|^t$ ← lifetime of the agent.

→ too huge table and impossible to be comprehensive!

→ The AI wants to produce rational behaviour from fairly small program rather than from a vast table!

(1) SIMPLE Reflex AGENT

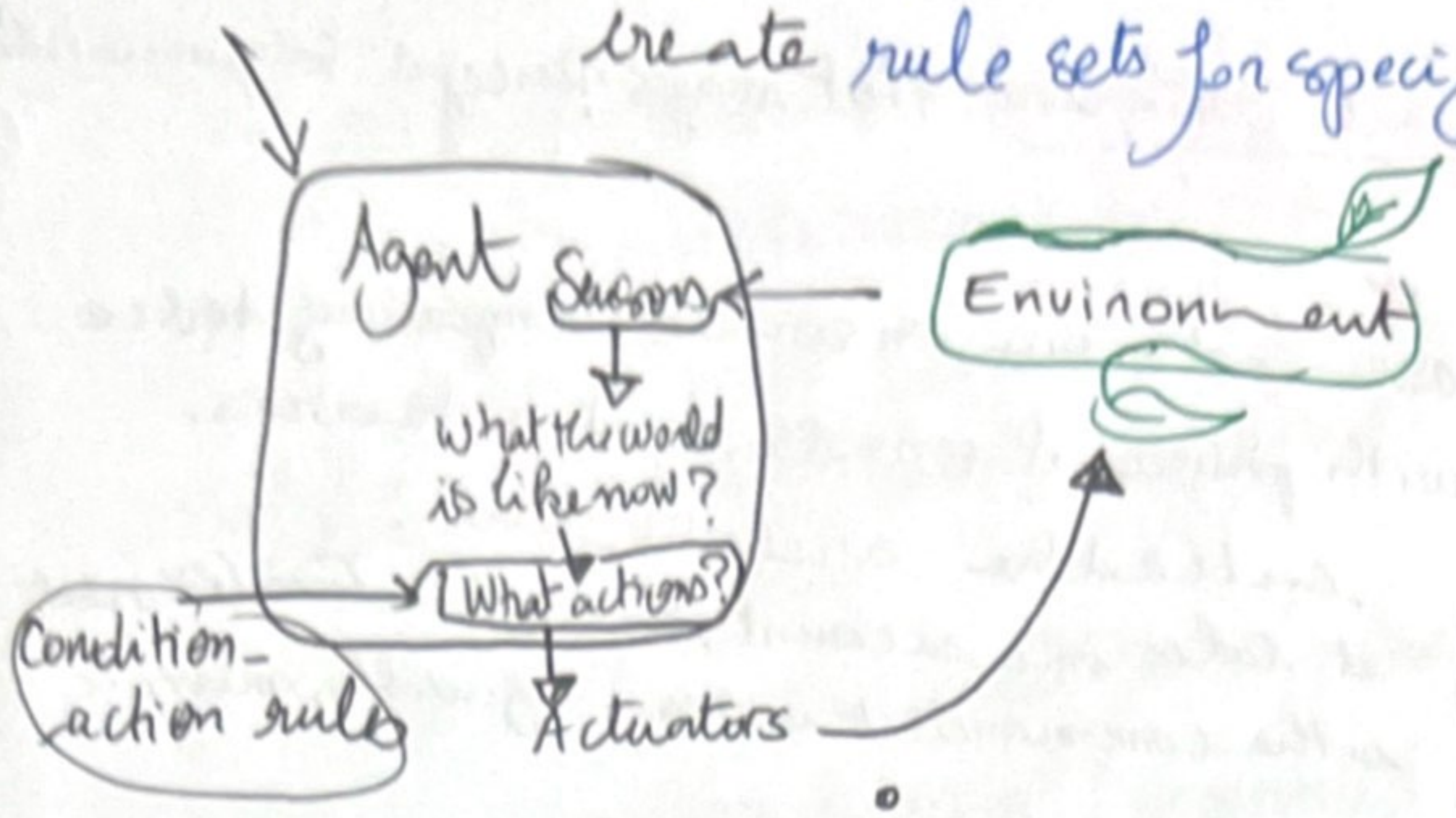
reflexes are common for humans, wether learned or "natural" (such as blinking when something approach the eye).

- Simplest type of agents
- on the basis of the current percept ignoring the rest of the percept history.

→ in the example: Vaccum cleaner

- Reduce from 4^T possibilities to 4
- Simple reflex behaviours occur even in more complex environments.
- Some percepts can trigger some established connection in the agent program to some action. Condition-action rule.

- More general/flexible approach is : first to build a general-purpose interpreter for condition-action-rules, and then to create rule sets for specific task environments.



- A simple reflex agent, acts according to a rule whose condition matches the current state as defined by the percept.

- They are simple but with Limited intelligence!

function SIMPLE-REFLEX-AGENT (percept) returns an action.
state → Interpret_Input.
rule → Rule-Match(state, rule)
action → rule.action
return action

- persistent: rules, a set of condition-action rules.

- Work only if the correct decision can be made on the basis of only the current percept. (only if the environment is fully observable).

→ Little unobservability → SERIOUS Trouble!

partially obsrv. → ∞ Infinite Loops are often unavoidable!

→ ∞ Solution Randomize its actions.

Fact: → Randomized simple reflex agent might outperform a deterministic simple reflex agent!

(Paper rock scissors.)

- Randomized behaviour is Rational in multiagent environments, unlike single-agent environments!

Not always enough to know what to do.

② Model Based reflex agents

- for handling partial observability. (we need to keep track of the part of the world it can't see now)

Eg: for braking keep previous frame from braking front car.

- Maintain some sort of **internal state** that depends on the percept history and reflects at least some of the un observed aspects of the current state.

updating internal State: it needs:

Model of the world

1- Some infos about how the world evolves independently of the agent.
2- Some info's about how the agent's own actions affect the world.

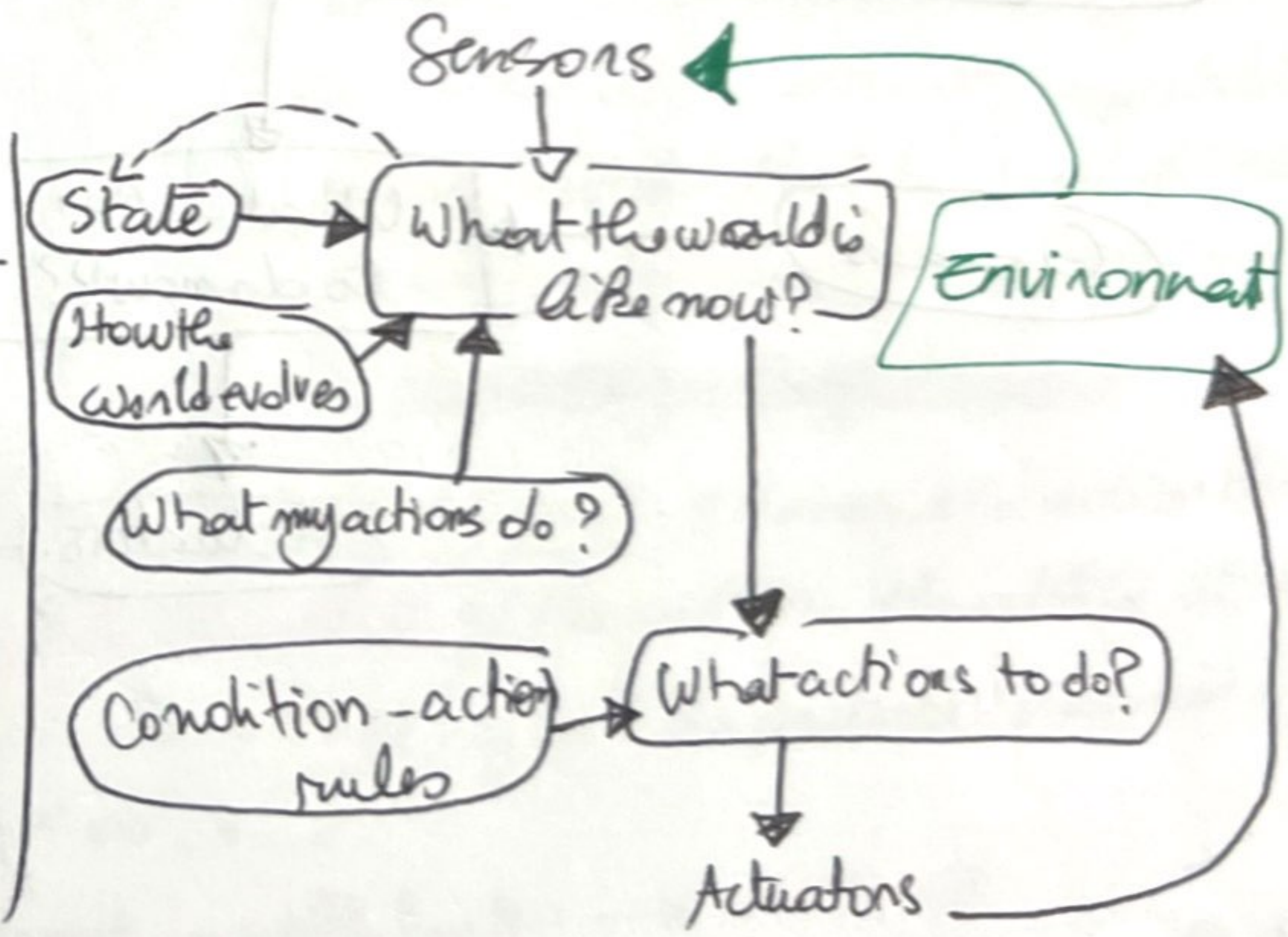
↳ any agent that uses such a model is called **model-based agent**.

- The details of representations of how models and states vary a lot depending on the type of environment and the particular technology used in the agent design.

- Independently, an agent can usually not determine the current state of a partially observable environment exactly.

↳ "Best Guess(es)" to represent the current state

- persistent : state
  - model
  - rules
  - actions



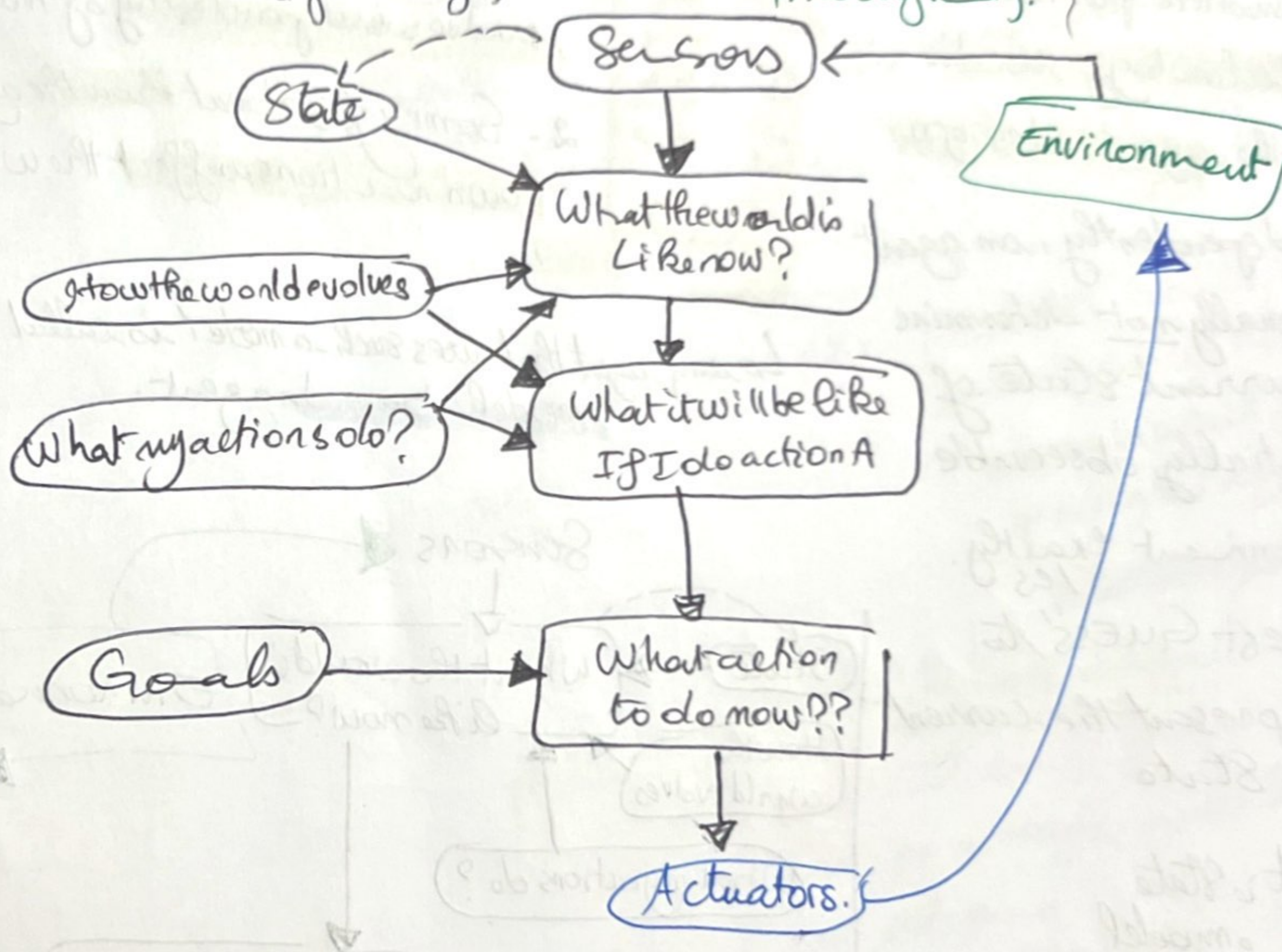
(3) Goal based agents:

- Agent needs some sort of goal description/info that describes situations that are desirable
  - Goal information can be combined with the model as seen in model-based reflex agents.
  - Sometimes goal-based action selection is straight forward.

selected actions → lead to goal satisfaction.

Often! • there is a need for a long sequence of actions to achieve the goal (sequence of sub-goals)

- It may/can be less efficient But MORE FLEXIBLE. (because the knowledge that supports its decisions is represented explicitly and can be modified).



(4) Utility-based agents: Goals are not enough to get HIGH Quality agent behaviour!

- How satisfied the agent is with the achievement of the goal?
- because there are various action sequences to reach a destination.

NEED for: MORE GENERAL performance measure. Utility — agent knows about it

- An agent's utility function is an internalization of the performance measure
  - > If internal utility function and External performance measure (agent does not know about it) are in agreement then the agent maximizes its utility and it will be RATIONAL with external performance measure.

++ Many advantages in terms of flexibility + learning.

2 cases

[Goals are inadequate (not enough) > but this agent can still make rational decisions.

(1) Conflicting goals: utility function specifies the appropriate tradeoff.

(2) SEVERAL Goals, where none of them can be achieved with certainty. → utility provides a way in which the likelihood of success can be weighed against the importance of goals.

happy/?

Success > Importance

* Rational Utility based agent: Chooses the action that maximizes the expected utility of the action, the utility it expects to derive, on average, given the probabilities and utilities of each outcome.

→ Rational agent → utility function → maximize.

- Uses a model of the world along with utility function that measures its preferences among states of the world.

Necessary because the percepts provide no indication of the agent's success.

Critic

⑤ Learning agents

- Learning element: makes improvements of agent (modifying its knowledge).
  - can make changes to any of the "knowledge" components in the agents diagram.
- Performance element: selects external actions.
  PE
  (takes in percepts and decides on actions).

Feedback on how well the agent is doing with respect to a fixed performance standard and it determines how the PE should be modified to do better in the future.

- Problem generator: explore more to ⤴

Suggests actions that will lead to new and informative experiences.

- exploiting Suboptimal actions actions in the short run might lead to discovery of much better actions for the long run.
- The generator's job is to suggest these exploratory actions.

Example in slide 63

Automated Taxi
Critic: rude language by other drivers.

▲ Learns how the world evolves from percept of sequences and observation of pairs of successive states of the environment.

- identify what to be improved.
- suggests experiments.

- Observation of the results of its actions → allow him to learn
- Learning is more difficult when: partially observable

▲ No need to access the external performance Standard!

Learning intelligent agents:

- Process of modifications of each component of the agent to bring the components into closer agreement with the available feedback information, Improving OVERALL PERFORMANCE!

How the components of agent programs work ???

Environment Representation

① atomic → ② factored → ③ Structured.

increasing complexity. → Increasing Expressiveness

① Atomic representation:

Each state of the world is indivisible → NO INTERNAL STRUCTURE.

o name of city during a route to a location.

② Factored representation:

needs more details.

Splits up each state into a fixed set of variables / attributes each of which can have a value.

- Finer + more expressive
- Uncertainty can be represented

Remark: two different atomic states cannot share anything in common, unlike the factored, two different factored states can share some attributes. (difference between logic propositions and variables)

③ Structured representation:

underlie relational databases and first-order logic, first order probability models, knowledge based learning and much of natural language understanding.

- Various areas of AI are based on factored representations including Constraint satisfaction algorithms, propositional logic, planning, Bayesian networks.

Conclusion page 70

↑ Expressiveness ↑ more can be captured and at least as concisely.

Remark: Sometimes several representations used at once!

General Tree Search Algorithm:

function Tree-Search (problem) return solution or failure.
initialize: frontier to the initial state of problems
loop:
if (frontier is empty) return failure.
choosing leaf node from the frontier.
→ if (node contains goal state?
→ return solution.
- expand the resulting node and adding the resulting nodes to the frontier.

FIFO queue
stack.
Priority queue using g(n) or f(n).

the difference in the graph:

- We'll have the explored set
- by initializing it to empty
- check if not in the frontier or explored set then expand the node. choosen from adding its resulting nodes to the frontier

* in the rectangular grid problem:

- 4^d leaves with repeated states
- $2*d^2$ distinct states within d steps of any given state.